Assignment 1 - PS12 - Optimal Planner

Group ID: G108

Team members: ARTHIKA G.; SRINEVEDA R S.; ASWATHY H.; AMIYA PALAI.; KUTAREKAR NISHCHAL AJAY.

1. Problem Definition

The problem is to plan a route for a delivery robot in a static 5 x 5 warehouse grid. The robot starts at S = (0, 0), must reach G = (4, 4), and must avoid blocked cells marked X. The allowed moves are Up, Down, Left, and Right. Diagonal movement is not allowed. Each valid move has cost 1.

The blocked cells in the supplied grid are (0, 3), (1, 1), (1, 3), (2, 1), (3, 2), and (3, 3). The search uses Local Beam Search with beam width k = 2.

2. PEAS Definition

Performance measure: Reach the goal from the start, avoid obstacles and invalid moves, minimize total path cost, and print a clear search trace with selected beam states and heuristic values.

Environment: A fully observable, deterministic, static 2D warehouse grid containing free cells, blocked cells, one start state, and one goal state.

Actuators: Move Up, Move Down, Move Left, and Move Right.

Sensors: The robot reads its current position, grid boundaries, blocked cells, start position, and goal position from the input grid.

3. Heuristic And Cost Functions

The heuristic function is Manhattan Distance:

```
h(n) = |goal_row - current_row| + |goal_column - current_column|
```

This heuristic is appropriate because the robot can move only horizontally or vertically. It gives the minimum number of moves needed to reach the goal in an obstacle-free grid.

The cost function assigns cost 1 to every valid move. Therefore:

```
g(n) = number of moves from S to n
total path cost = number of moves in the final route
```

For the given grid, the Manhattan lower bound from S = (0, 0) to G = (4, 4) is 8. The implemented search finds a route of cost 8, so the route is optimal for this grid.

4. Algorithm Design

The program implements Local Beam Search with k = 2. The initial beam contains only the start node. At every iteration, successors are generated from all states in the current beam using the move order Up, Down, Left, Right. Invalid successors are removed if they are outside the grid, blocked by X, or already present in the same path.

Duplicate states are removed by position. If the same position is reached through more than one path, the implementation keeps the best node using the deterministic order: lower heuristic, lower path cost, lower row index, lower column index, and finally lexicographic path order. The best k states are inserted into a bounded BeamFringe and become the next beam. The search stops when the goal is selected in the beam or when no valid successor

remains.

For the given grid, the final path is:

```
(0, 0) -> (0, 1) -> (0, 2) -> (1, 2) -> (2, 2) -> (2, 3) -> (2, 4) -> (3, 4) -> (4, 4)
```

Total path cost = 8.

5. Data Structures And Fringe Design

SearchNode stores the current position, full path, path cost, and heuristic value. IterationRecord stores the trace for each iteration: current beam, generated successors, duplicate-free successors, and selected beam states. BeamFringe is a bounded queue-like structure with capacity k. Its insert operation raises a clear error when the fringe is full, and its delete operation raises a clear error when the fringe is empty. This satisfies the required data-structure error handling.

The grid is stored as a two-dimensional list. Candidate states are sorted using Python's stable sorting, and duplicates are removed using a dictionary keyed by grid position.

6. Complexity Analysis

Let k be the beam width, b be the branching factor, and d be the depth of the solution. In this problem, b <= 4 because only four actions are possible. At each level, at most k states are expanded and up to k*b successors are generated. The successors are sorted to select the best k states.

Time complexity: O(d * k * b log(k*b)).

Space complexity: O(k * d), because each selected beam node stores its path. With fixed k = 2 and b <= 4, the search is memory efficient compared with exhaustive search.

7. Alternate Modeling Approach

An alternate way to model the problem is as a graph search problem solved using A* Search. Each free grid cell becomes a node, and edges connect valid four-directional neighbors with cost 1. A* uses f(n) = g(n) + h(n), where g(n) is the path cost so far and h(n) is Manhattan Distance. With this admissible heuristic, A* can guarantee an optimal path if one exists, but it may require more memory because it maintains larger open and closed lists. Local Beam Search uses less memory by retaining only k promising states at each level, but it can prune alternatives aggressively in harder grids.

8. Implementation Notes

The Python file is modular and reads the grid from inputPS12.txt, so the grid is not hardcoded. The output trace is written to outputPS12.txt and printed to the console. Basic error handling is included for invalid input files, wrong dimensions, missing start or goal states, invalid grid symbols, invalid k values, full fringe insertion, and empty fringe deletion.